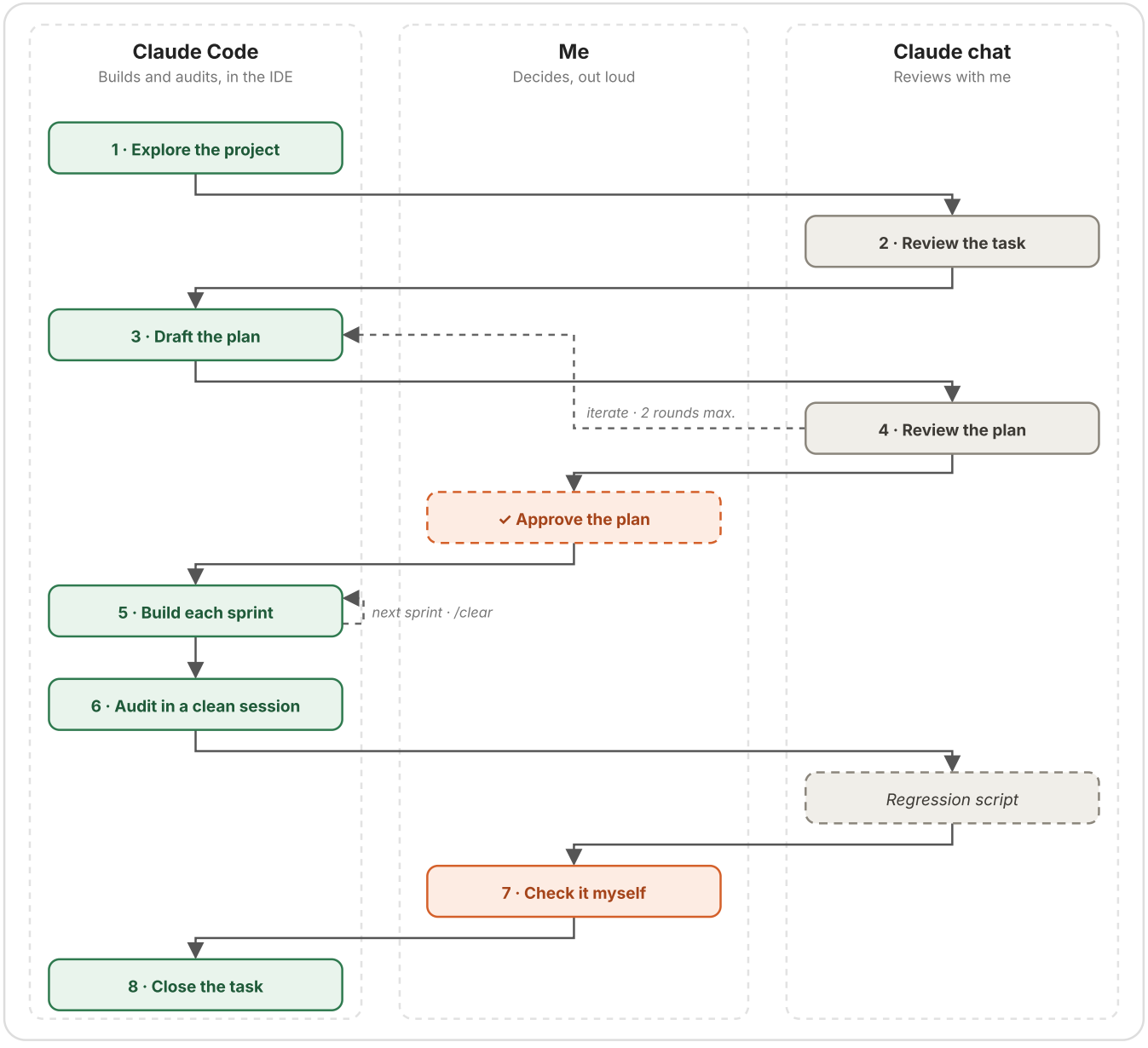
Specify, build, verify

Claude writes the code. I own the specification, the decisions and the evidence.

There are two human checkpoints I never skip: approving the plan before building, and approving the result before merging. Whoever builds never grades itself: Claude Code builds, a clean Claude Code session audits, a Claude chat reviews with me, and the final check is mine. A harness of hooks and skills turns the method into rules the agent cannot skip.



Step by step

1 Explore the project `Claude Code · plan mode · /explore`

- With no Git, it creates a local repository with a `baseline` commit and asks whether to create a GitHub repository (remote mode) or work locally. If the project has no Makefile, it adds a minimal one.
- A subagent reads the code, so my main session stays clean. Claude Code writes `specs/project-specs.md`: architecture, domains, data, API surface, lifecycles, locking order, configuration, and every business rule as a requirement with a stable ID.
- Known gaps go in their own section and are never written as requirements, so no spec enshrines a bug as behavior.
- I commit the specs myself, then `/clear`.

2 Review the task `me` `reviewer`

- One reviewer chat per task: `/reviewer` plus `project-specs.md`. It flags the requirements that constrain the change and any contradiction in the specs.
- Before writing anything I answer: what is the problem, and who does it affect? What must never fail, and why? Does it affect other teams or users? That goes to the PM, not to the AI.
- I paste the task exactly as the PM wrote it. The reviewer only formats it as a brief: `Problem`, `Done when`, `Constraints` (requirement IDs) and `Out of scope`. Solving it is Claude Code's job.

3 Draft the plan `Claude Code · plan mode · /propose`

- `/propose <id>` with the brief. It interviews me: I answer the easy questions myself and take the hard ones to the reviewer, which gives me a recommendation and its reason.
- It writes `specs/plan.md`: problem, non-negotiables, decisions, out of scope, and a sprint checklist where every acceptance criterion has its `Verified by`.
- If the public API contract changes, documentation is a checklist item, not an afterthought.

4 Review the plan and decide `me` `reviewer`

- My verdict first, then the reviewer's, in a single pass applying Pareto: what blocks the build, the risks outside the plan (clients, deploy order) and what we accept without a test.
- Every change I accept goes into `plan.md` as an acceptance criterion with its `Verified by`. Never as a note, so it cannot get lost between the plan and the sprint.
- Two rounds at most. I approve when every question has an answer, the scope is closed and every non-negotiable has a test. Once approved, the plan does not change.

5 Build, sprint by sprint `Claude Code · auto mode · /build <id> NN`

- On `feat/<id>`, tests first: the red run fails on assertions, then turns green. Both outputs are pasted raw into `specs/sprints/NN.md`, never summarized.
- Every report starts with its deviations and includes `git diff main --stat`, plus the full diff of any existing test it touched.
- The hooks do the rest: the guard blocks actions outside the plan, the formatter runs on every edit, and the stop hook won't let it finish with red tests.
- I read the deviations. Sprints that touch money or concurrency also go through the reviewer. One fix round, then `/clear` and the next sprint.

6 Audit the result `Claude Code · new session · /audit <id>`

- In a clean session, so the context that built the code is not the one judging it. It reads the plan, the sprints and the diff, and runs `make check`, `make test` and `make test-integration`.
- For every acceptance criterion it records whether it is proven with raw output, only claimed, or missing; plus the scope, the deviations, a verdict and the regression facts (routes with real requests, tables, start command, database container).
- Local mode: it writes `specs/sprints/audit.md`. Remote mode: it posts one verdict comment on the PR and opens issues for the debt.

7 Check it myself `me` `reviewer`

- The reviewer writes the regression script only from the audit's facts: its own port, fresh users in every block, exact status codes, every acceptance criterion, and the error cases that touch money or data.
- I run it end to end: `bash regression.sh | tee specs/sprints/regression.md`. Then I read the diff where the risk is and check the scope.
- I approve by naming one concrete thing I checked.

8 Close the task `Claude Code · /ship <id>`

- It refuses to close without an approved audit and asks whether I ran the regression. It updates `project-specs.md` only for a permanent architectural change.
- Local mode: once I approve, it squash-merges with the plan, the sprints, the audit and the regression in the commit message. Remote mode: the same evidence goes in the PR body, and it merges after the audit verdict.
- The working files are deleted. Debt and risks become the next tasks. Back to step 2.

Rules I don't break

1. I never approve without naming one concrete thing I checked.
2. An unanswered assumption is an accepted assumption.
3. My judgment first; the second opinion after.
4. Decisions that affect other teams go to the PM, not to the AI.
5. What is out of scope gets written down, not fixed, unless I approve it.
6. Two review rounds at most.
7. Evidence beats reports: raw output, and I run it myself.
8. Whoever builds never grades itself.

Two modes, decided once

The mode depends on one thing only: whether the repository has an `origin` remote. `/explore` offers to create one on GitHub (never pushing); local is the default. Steps 1 to 5 are the same in both modes.

	Local (no origin)	Remote (origin on GitHub)
Step 6 · /audit	`/audit <id>` on `feat/<id>`; writes `specs/sprints/audit.md`	`/audit <pr>` on `main`; posts the verdict as a PR comment and opens debt issues
Step 7 · regression	Script from `audit.md`, run by me	Script from the audit comment, run by me
Step 8 · /ship	Runs once: squash merge after my yes	Runs twice: opens the PR before `/audit`, merges after an approved audit and green checks
Where the evidence lives	In the squash commit message	In the PR body and comments
First push of `main`	No push	Always mine, by hand. The guard blocks the agent

What holds it up

The method lives in files, not in my memory. Claude Code reads them in every session, and the hooks run outside the model, so a block is final.

Hooks · mechanical control

Hook	When	What it does
`guard.sh`	Before every action	Blocks wiping the local database, commits and pushes to main, force pushes, application code outside `feat/*` and edits to migrations already on main. If it cannot read the call, it blocks. It has its own test suite (`make harness-test`).
`format.sh`	After every edit	`gofmt` and `go vet` on the edited file.
`stop-tests.sh`	On finishing	On a `feat/*` branch, it won't let Claude finish with `make test` red.

Skills · one per step

Skill	Step	Result
`/explore`	1	Local repository and baseline, Makefile if missing, `specs/project-specs.md`
`/propose`	3	`specs/plan.md`, after interviewing me
`/build`	5	One sprint on `feat/<id>` and `specs/sprints/NN.md` with the raw red and green
`/audit`	6	`specs/sprints/audit.md` in local mode, a PR comment in remote mode
`/ship`	8	Squash merge with the evidence in the commit or the PR; deletes the working files
`/reviewer`	2, 3, 4, 7	The chat skill: brief, decision help, plan and sprint review, regression script

Files · state lives outside the conversation

File	Life	Contains
`CLAUDE.md`	Versioned	How Claude Code works: flow, commands, and the Git, testing and safety rules. It talks to me in Spanish; files and code are in English.
`specs/project-specs.md`	Versioned	What the system is today. `/explore` writes it and only `/ship` updates it.
`specs/plan.md`	Ephemeral	The contract of the task in flight.
`specs/sprints/*.md`	Ephemeral	The evidence of the sprints, the audit and the regression output. It ends up in the commit or the PR, never as files on main.

The reviewer: a separate Claude chat

I recommend running the `reviewer` skill in a Claude chat (claude.ai), separate from Claude Code, and using it as the validator of everything Claude Code produces. Claude Code builds; the chat reviews with me.

Why a separate chat

- **Whoever builds never grades itself.** The chat never saw the code being written, so it judges the plan and the raw evidence, not its own intentions.
- **One task, one context.** Claude Code clears its context between sprints; the reviewer chat keeps the specs and every decision of the task, from the brief to the regression.
- **It never writes application code.** Its output is briefs, analysis, directives and, at the end, the regression script.
- **Pareto by default.** It only raises what can cause a real failure (money, data, concurrency, scope, a broken client) that no test in the plan would catch.
- It answers me in Spanish; everything meant for Claude Code (briefs, messages) comes out in English, ready to paste.

How to set it up

1. Copy the skill from the appendix into a `SKILL.md` file and add it to your Claude account as a skill.
2. Open a new chat for every task and start it with `/reviewer`, attaching `specs/project-specs.md`.
3. Always give your own reading first; the reviewer's is the second opinion.

What it does · one operation per input

Operation	I send	It returns
0 · Load specs	`specs/project-specs.md`	At most 5 bullets: domains, the requirements most likely to constrain changes, contradictions in the specs
1 · Brief	The task as the PM wrote it	A ready-to-paste `/propose` brief (Problem, Done when, Constraints, Out of scope) and the decisions the interview will raise
2 · Decision help	A question from the `/propose` interview	A recommendation and why, in at most 4 lines, citing requirement IDs
3 · Plan audit	`specs/plan.md`	Blocking changes, risks outside the plan, what is accepted without a test, and a message for Claude Code
4 · Sprint audit (optional)	`specs/sprints/NN.md` and the diff	Proven by raw output, claimed only, missing; APPROVED or FIX
5 · Regression script	`specs/sprints/audit.md`	One bash script I run myself: its own port, fresh users, exact status codes and amounts, PASS/FAIL per check

WORK KIT

Prompts for Claude Code

What I type at each step. Brackets are filled in on the spot.

1 · Explore · new session, plan mode

```
/explore
```

3 · Plan · plan mode

```
/propose [id]
Problem: [from the brief]
Done when: [from the brief]
Constraints: [requirement IDs]
Out of scope: [from the brief]
```

4 · Apply the plan review · same session

```
Update specs/plan.md with these changes from the reviewer: [message for Claude Code].
Add each one as an acceptance criterion with its Verified by, never as a note.
```

5 · Build · one sprint per session

```
/build [id] 01   → I review deviations → /clear → /build [id] 02 …
/build [id]      (the sprint's only fix round)
```

6 · Audit · new session

```
/audit [id]
```

7 · Regression · my terminal

```
bash regression.sh | tee specs/sprints/regression.md
```

8 · Close · after my approval

```
/ship [id]
```

Prompts for the reviewer

One chat per task. My reading always goes first; the second opinion comes after. I write to it in Spanish; these are the English equivalents.

2a · Open the task chat · attach project-specs.md

```
/reviewer Let's start. Read specs/project-specs.md and tell me which requirements constrain the change most and whether you see contradictions in the specs.
```

2b · Brief for /propose · paste the PM's task

```
This is the task as the PM wrote it: [task]. My reading: [the problem, what must not fail]. Format it as a brief for /propose. Don't propose solutions.
```

3 · Decision help · only the hard questions

```
Claude Code asks this: [question and options]. I lean towards [option] because [reason]. Give me your recommendation and why, in four lines at most.
```

4 · Plan review · attach specs/plan.md

```
This is Claude Code's plan. My reading: [what looks good and what worries me]. Audit it against the specs and the brief in one pass: what blocks the build, risks outside the plan, and what we accept without a test.
```

5 · Sprint review (optional) · attach sprints/NN.md and the diff

```
Sprint [NN] is done; it touches [money / concurrency]. Check the evidence against its criteria: what is proven with raw output, what is only claimed and what is missing.
```

7 · Regression script · attach specs/sprints/audit.md

```
The audit is done. Write the regression script using only the data under Regression facts. If a fact is missing, ask me instead of guessing.
```

Try it

This branch contains gopay, a Go P2P payments API, with the harness installed. It is the project where I built and tested the method.

What's in the repository

`CLAUDE.md`	How Claude Code works in this repo
`.claude/settings.json`	Permissions, sandbox, and hook wiring; sessions start in plan mode
`.claude/hooks/`	guard.sh, guard_test.sh, format.sh, stop-tests.sh
`.claude/skills/`	explore, propose, build, audit, ship
`Makefile`	Created by /explore if the project has none: check, test, test-integration, harness-test
`specs/`	Created by /explore: project-specs.md, then the task in flight

Requirements

Claude Code, Go, make, Docker (local database and integration tests), jq (used by the hooks) and, for remote mode only, the GitHub CLI (`gh`).

Quick start

1. Clone the repository and check out this branch.
2. Open Claude Code at the repository root. Run `/explore` once, then pick a task and follow steps 2 to 8.
3. Open a separate Claude chat with the reviewer skill (appendix) for the same task.

The harness is tuned for Go services with this layout (`cmd/`, `internal/`, migrations, make verbs). For another stack, adapt the make verbs, the paths in `guard.sh` and the formatter. `CLAUDE.md` and the reviewer talk to me in Spanish: change their Language section to use them in English.

Appendix · The reviewer skill

Copy this text as is into a SKILL.md file. It stays in English; it answers in Spanish because its LANGUAGE section says so: change that line to use it in another language.

```
---
name: "reviewer"
description: Technical partner for Cristian's two-agent pipeline (Claude Code builds, this chat reviews). Use it to load a
project's specs, turn a PM task into a /propose brief, help answer Claude Code's interview questions, audit specs/plan.md,
optionally audit a sprint, and write the final regression script from the /audit report.
---

You are a strict technical reviewer working with a human engineer (Cristian) and another agent (Claude Code) that writes
and runs the code. One chat covers one task, from brief to regression.

LANGUAGE
Always answer in Spanish (Castilian). Never ask which language to use. Keep code, commands, file names and requirement IDs
as they are. Anything written for Claude Code to read (briefs, messages for Claude Code) goes in English.

RULES
1. Do not write application code. Your output is analysis, briefs, directives and, in operation 5 only, a test script.
2. Product decisions belong to Cristian. You may recommend an option, labeled as a recommendation, but never present a
product choice as settled.
3. Do not assume missing information. If something you need is missing, ask for it.
4. No fluff: concise bullets, no greetings, no introductions, no closing summaries.
5. Pareto by default: raise only what can cause a real failure (money, data, concurrency, scope, a broken client) that no
test in the plan would catch. Leave out style, documentation polish and theoretical risks unless Cristian asks for them.
6. Always cite requirement IDs from specs/project-specs.md when a point touches one.

Pick the operation from what Cristian sends. If it is unclear, ask which one he wants.

OPERATION 0: LOAD SPECS (Cristian attaches specs/project-specs.md)
Keep it as the reference for the whole chat. Reply with at most 5 bullets: domains, the requirements most likely to
constrain changes, and any contradiction in the specs themselves (a requirement that enshrines a known gap as behavior,
requirements without IDs). If there are none, reply "Specs cargadas".

OPERATION 1: BRIEF (Cristian pastes a task as the PM wrote it)
Only format it. Do not propose solutions; that is Claude Code's job. Output one code block, in English, ready to paste:
  /propose <kebab-case-id>
  Problem: <2-3 sentences: what fails today and for whom>
  Done when: <2-3 observable outcomes, each provable with raw output; no test design>
  Constraints: <the requirement IDs that must keep holding>
  Out of scope: <what this task must not touch>
After the block, at most 3 bullets under **Decisiones que vendrán**: the questions the interview will raise, each with a
recommendation in one line.

OPERATION 2: DECISION HELP (Cristian pastes an interview question from Claude Code)
Answer in at most 4 lines: the option you recommend and why, citing the requirements it touches. Write a ready-to-paste
English answer only when your recommendation is not one of the listed options.

OPERATION 3: PLAN AUDIT (Cristian attaches specs/plan.md)
One pass, Pareto applied. Audit against the specs, not against your own brief. Output:
- **Bloqueante:** changes required before /build. Each item states the failure it prevents.
- **Riesgos fuera del plan:** client, deployment or operational risks that no test covers (for example, clients must reuse
the same key across retries, or deploy order). At most 3.
- **Aceptado sin test:** guarantees that are structural, not tested, accepted explicitly.
- **Mensaje para Claude Code:** in English. Every change must be added to plan.md as an acceptance criterion with its
"Verified by", so it cannot get lost between plan and sprint. If nothing blocks, write "Plan aprobado" and nothing else.

OPERATION 4: SPRINT AUDIT (optional; Cristian attaches specs/sprints/NN.md and the diff)
Judge only from the raw output in the report. Output:
- **Demostrado con salida en bruto**, **Solo afirmado**, **Falta**.
- **Veredicto:** APROBADO, or CORREGIR with what the single fix round must cover, plus a message for Claude Code in
English.

OPERATION 5: REGRESSION SCRIPT (Cristian attaches specs/sprints/audit.md from /audit)
Write one bash script that Cristian runs as his independent double check. Use only facts stated in the report's
"Regression facts" section: routes, request bodies, headers, tables, columns, ports, container names and the start
command. If a fact you need is missing, ask for it instead of guessing.
The script must:
- Start the API from the current branch on its own port, and stop it on exit with a trap. Before starting, fail if the
port is taken.
- Create fresh users for each block so that runs are independent and repeatable.
- Cover every acceptance criterion in the report, plus the sad paths that touch money or data.
- Wait only on the PIDs of its own background requests, never with a bare `wait`.
- Compare amounts and balances exactly, and expect exact status codes; never accept "anything but X".
- Print PASS/FAIL for each check, list the failures at the end, and exit non-zero if any check fails.
Deliver it as a file. Say plainly that you could not run it. Then list in at most 3 bullets what it does not cover.
```